

- Description

This module manages an intelligent traffic signal system, ensuring that:

1. **Traffic lights adjust their timing dynamically** based on real-time traffic flow.
2. **Each intersection is monitored** to detect congestion and set optimal green light durations.
3. **A log of signal changes** is maintained for traffic analysis and optimization.

This system ensures efficient traffic flow by dynamically adapting signal durations based on traffic conditions

- Specification

module IntelligentTrafficControl

var greenTime, trafficDensity: integer

var signalLog: sequence of string

process updateGreenTime(trafficDensity: integer)

functional scenario form:

trafficDensity > 80 and trafficDensity <= 100 && greenTime = 45, High traffic!

Green time updated to 45 seconds.

||

trafficDensity > 50 and trafficDensity <= 100 && greenTime = 30, Moderate

traffic! Green time updated to 30 seconds.

||

trafficDensity <= 50 and trafficDensity >= 0 && greenTime = 15 Low traffic!

Green time updated to 15 seconds.

end process

end module

- Java Code

```
package code;
import java.util.ArrayList;
import java.util.List;
class TrafficSignal {
    private String intersection;
    private int greenTime;
    private List<String> signalLog;
    public TrafficSignal(String intersection, int defaultGreenTime) {
        this.intersection = intersection;
        this.greenTime = defaultGreenTime;
        this.signalLog = new ArrayList<>();
    }
    public void updateGreenTime(int trafficDensity) {
        if (trafficDensity > 80 && trafficDensity <= 100) {
            greenTime = 45;
        } else if (trafficDensity > 50 && trafficDensity <= 80) {
```

```

        greenTime = 30;
    } else if (trafficDensity > 0 && trafficDensity <= 50){
        greenTime = 15;
    } else {
        greenTime = 0;
    }
    if(greenTime == 0) {
        System.out.printf("invalide input%n ");
    }else {
        signalLog.add(String.format("Intersection %s: Green time update
d to %d seconds (Traffic: %d%%)",
            intersection, greenTime, trafficDensity));
        System.out.printf("Green light at %s set to %d seconds due to t
raffic density: %d%%\n",
            intersection, greenTime, trafficDensity);
    }
}
}
public void printSignalLog() {
    System.out.println("Traffic Signal Log for " + intersection + "
:");
    for (String log : signalLog) {
        System.out.println(log);
    }
}
}
public class c09_IntelligentTrafficSystem {
    public static void main(String[] args) {
        TrafficSignal signal = new TrafficSignal("Main St & 1st Ave", 1
5);
        // testcase-VT:
        //     signal.updateGreenTime(-21); // trafficDensity < 0
        //     signal.updateGreenTime(3); // trafficDensity > 0 && trafficDe
nsity <= 50
        //     signal.updateGreenTime(62); // trafficDensity > 50 && trafficD
ensity <= 80
        //     signal.updateGreenTime(87); // trafficDensity > 80 && trafficD
ensity <= 100
        //     signal.updateGreenTime(134); // trafficDensity > 100
        //     signal.printSignalLog();

        //testcase-FT:
        //     signal.updateGreenTime(61); // trafficDensity < 0
        //     signal.updateGreenTime(27965); // trafficDensity > 0 && traff
icDensity <= 50

```

```
//      signal.updateGreenTime(9006397); // trafficDensity > 50 && trafficDensity <= 80
//      signal.updateGreenTime(35181); // trafficDensity > 80 && trafficDensity <= 100
//      signal.updateGreenTime(96); // trafficDensity > 100
//      signal.printSignalLog();
//      // testcase-Z3:
//      signal.updateGreenTime(100); // trafficDensity < 0
//      signal.updateGreenTime(79); // trafficDensity > 0 && trafficDensity <= 50
//      signal.updateGreenTime(101); // trafficDensity > 50 && trafficDensity <= 80
//      signal.updateGreenTime(79); // trafficDensity > 80 && trafficDensity <= 100
//      signal.updateGreenTime(101); // trafficDensity > 100
//      signal.printSignalLog();

//      // testcase-UVT:
//      signal.updateGreenTime(-21); // trafficDensity < 0
//      signal.updateGreenTime(3); // trafficDensity > 0 && trafficDensity <= 50
//      signal.updateGreenTime(62); // trafficDensity > 50 && trafficDensity <= 80
//      signal.updateGreenTime(87); // trafficDensity > 80 && trafficDensity <= 100
//      signal.updateGreenTime(134); // trafficDensity > 100
//      signal.printSignalLog();
    }
}
```

- Coverage report

1. Test case number: 5
2. V-Method testing branch coverage: 100%

Element	Coverage	Covered Branches	Missed Branches	Total Branches
✓ VibrationTestingTool	1.5 %	12	812	824
✓ src	1.5 %	12	812	824
✓ code	1.5 %	12	812	824
✓ c09_IntelligentTrafficSystem.java	100.0 %	12	0	12
> c09_IntelligentTrafficSystem		0	0	0
> TrafficSignal	100.0 %	12	0	12

3. Vibration testing instruction coverage: 98.0%

Element	Coverage	Covered Instructions	Missed Instructions	Total Instructions
✓ VibrationTestingTool	0.9 %	148	15,452	15,600
✓ src	0.9 %	148	15,452	15,600
✓ code	0.9 %	148	15,452	15,600
✓ c09_IntelligentTrafficSystem.java	98.0 %	148	3	151
> c09_IntelligentTrafficSystem	88.9 %	24	3	27
> TrafficSignal	100.0 %	124	0	124

4. Fuzz testing branch coverage: 81.2%

Element	Coverage	Covered Branches	Missed Branches	Total Branches
▼ VibrationTestingTool	1.6 %	13	815	828
▼ src	1.6 %	13	815	828
▼ code	1.6 %	13	815	828
▼ c09_IntelligentTrafficSystem.java	81.2 %	13	3	16
> TrafficSignal	81.2 %	13	3	16
> c09_IntelligentTrafficSystem		0	0	0

5. Fuzz testing instruction coverage: 95.4%

Element	Coverage	Covered Instructions	Missed Instructions	Total Instructions
▼ VibrationTestingTool	0.9 %	144	15,456	15,600
▼ src	0.9 %	144	15,456	15,600
▼ code	0.9 %	144	15,456	15,600
▼ c09_IntelligentTrafficSystem.java	95.4 %	144	7	151
> TrafficSignal	96.8 %	120	4	124
> c09_IntelligentTrafficSystem	88.9 %	24	3	27

6. Z3 branch coverage: 81.2%

Element	Coverage	Covered Branches	Missed Branches	Total Branches
▼ VibrationTestingTool	1.5 %	13	827	840
▼ src	1.5 %	13	827	840
▼ code	1.5 %	13	827	840
▼ c09_IntelligentTrafficSystem.java	81.2 %	13	3	16
> TrafficSignal	81.2 %	13	3	16
> c09_IntelligentTrafficSystem		0	0	0

7. Z3 instruction coverage: 95.4%

Element	Coverage	Covered Instructions	Missed Instructions	Total Instructions
▼ VibrationTestingTool	0.7 %	144	21,949	22,093
▼ src	0.7 %	144	21,949	22,093
▼ code	0.7 %	144	21,949	22,093
▼ c09_IntelligentTrafficSystem.java	95.4 %	144	7	151
> c09_IntelligentTrafficSystem	88.9 %	24	3	27
> TrafficSignal	96.8 %	120	4	124

8. Unimproved V-method branch coverage: 100%

Element	Coverage	Covered Branches	Missed Branches	Total Branches
▼ VibrationTestingTool	1.5 %	12	812	824
▼ src	1.5 %	12	812	824
▼ code	1.5 %	12	812	824
▼ c09_IntelligentTrafficSystem.java	100.0 %	12	0	12
> c09_IntelligentTrafficSystem		0	0	0
> TrafficSignal	100.0 %	12	0	12

9. Unimproved V-method instruction coverage: 98%

Element	Coverage	Covered Instructions	Missed Instructions	Total Instructions
▼ VibrationTestingTool	0.9 %	148	15,452	15,600
▼ src	0.9 %	148	15,452	15,600
▼ code	0.9 %	148	15,452	15,600
▼ c09_IntelligentTrafficSystem.java	98.0 %	148	3	151
> c09_IntelligentTrafficSystem	88.9 %	24	3	27
> TrafficSignal	100.0 %	124	0	124